

The 9th International Conference on Emerging Ubiquitous Systems and Pervasive Networks (EUSPN 2018)

Design and Implementation of an End-to-End Web based Trusted Email System

Muhammad Zeeshan Sabir^{a,*}, Muhammad Yousaf^a

^aFaculty of Computing, Riphah International University, Islamabad (46000), Pakistan

Abstract

Web based email systems are important services on internet that store and route emails between senders and recipients. In principle, these are intermediate systems that have the potential of intercepting the end user's emails. Privacy of emails is often compromised by law enforcement agencies as well as by email service providers for user profiling and targeted advertisements. Many solutions have been proposed to solve such privacy concerns by means of end-to-end email encryption. However, these solutions pose other challenges to maintain usability, portability and trusted key management on web. These systems maintain and distribute end-user cryptographic keys through their web application. Unfortunately, key generation and sharing mechanism is often un-trusted and kept hidden. Trust is directly dependent on web application implementation and can be manipulated by means of key modification through unannounced change of source code (backdoors) on server. In order to address these issues, we propose a new PKI based architecture for a trusted web-based email system. It offers significant usability and portability. Moreover, all the cryptographic operations are open and transparent to end users.

© 2018 The Authors. Published by Elsevier Ltd.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)
Selection and peer-review under responsibility of the scientific committee of EUSPN 2018.

Keywords Email Privacy; Email Privacy and Trust; End-to-End Privacy Architecture; Secure Email Systems

1. Introduction

Email is one of the oldest applications on the Internet. A web based email system consist of a web application hosted on HTTP server and web client like web browser to compose, send and receive an email. Because of the storage capacity offered by service providers, many people communicate using free email providers. Mostly, people keep emails in form of personal diary and rarely deletes their email, and keep their personal data on provider's servers for years [1].

* Corresponding author. Tel.: +92-312-515-2272; fax: +92-321-600-7922

E-mail address: zeeshan.sabir@riphah.edu.pk

Comparatively, email accounts contain more sensitive data in nature than other applications like social media [2]. For an attacker, a hacked email account can leak personal information of individuals. It also allows them to access many of victim's online services. Common users trust on service providers for the privacy of their email messages. However, this trust is often misused in name of targeted advertisements [3]. Email providers operate under policy where they are required to share users' email data with law enforcement agencies [4,5]. Moreover, attackers often attack and compromise large number of email accounts [7,8].

Defending against the above mentioned attacks has always been challenging and difficult which requires sophisticated knowledge of key management and configuration to send or receive a secure email. In automatic key management systems, it is also challenging because it requires significant changes in email infrastructure and email clients. Existing web based email service providers provide secure emails using Transport Layer Security (TLS) as shown in the Fig.1 (a). However, these solutions only secure eavesdropping from external attackers on the communication path between email clients and servers. It doesn't protect from internal attacks on the server. As compared to this, end-to-end secure systems provide protection from the email service provider's end too as shown in Fig.1 (b).



Fig. 1. (a) Example showing lack of end-to-end privacy in Gmail; (b) Example of an end-to-end privacy in email systems

2. Related Work

This section presents some existing solutions to improve the privacy concerns of the emails on provider's server. In general, these systems differ in terms of trust model and design properties such as usability, portability and scenario supported.

2.1. Proxy-Based Designs

Proxy-based techniques use their servers to encrypt and decrypt the messages and could not be considered truly as end-to-end secure. Brown et al [9] introduced a proxy based architecture. The system was an extension of an email client that requires much technical knowledge to operate and was less portable. In EmailCloak [12] end crypto keys are not handled at end-users e.g. a bank, before sending email directly to Alice, encrypts at its EmailClock MTA with Alice's public key. Gmail in-between cannot read email as shown in Fig.2. This mechanism is not truly end to end because of fact that it only protects email on targeted servers. Jammal et al [13] proposed a proxy-based design that required to use an extra hardware (mobile phone) to execute secure email operations. Another Windows-based system Opaque-Mail communicate with the mail clients and requires local installation on all users' devices. Similarly, by design proxy re-encryption does not have significant impact on email privacy Furthermore, users trust could be manipulated by planting backdoor through application source code.

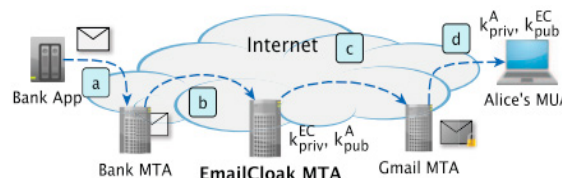


Fig. 2. Proxy design protecting email privacy only at targeted MTA

2.2. Identity-based Encryption (IBE)

In IBE, user's email address is represented as public key, and private keys are generated by a centric party known as Private Key Generator (PKG). After basic authentication, private keys are transmitted through PKG to end users on the secure channel. It is known that IBE schemes suffer from the key theft problem allowing the PKG to decrypt any message [21]. IBE approach is used by service providers like Voltage email security system [22], MessageGuard and FortiMail developed by Fortinet.

2.3. Plugins based approaches

Emails are also protected through various plugins. Mailvelope [25] uses manual key management requiring users to distribute and manage keys manually that affects usability for a novice user. Plugins like Jumble mail and Secure Gmail [27] are PGP based and encrypt messages at their own managed keys, so end user has to trust on provider. Route et al [17] studied PGP with Mailvelope plugin. Even after significant improvement in Mailvelope security, still, it has low usability and hard to operate for common users without knowledge of public key cryptography. The practicality of PGP/SMIME is addressed [24] in detail. Solutions like Private Webmail, Virtu, and Xmail generate and distribute encryption keys at their servers by hiding key management process. These solutions are paid and unavailability of source code raises trust issues for end users.

2.4. PKI based solutions

PKI based systems are based on asymmetric cryptography and uses PGP for end to end data encryption. PGP has usability issues related to key management discussed by Ruoti et al [17] and Alma Whitten [18]. In PKI, the service provider itself acts as certificate authority (CA) and is responsible for maintaining digital certificates and private keys for the end users. Also, these systems have to keep balance between privacy, portability and usability on web. Lavabit [26] claimed to provide end to end email privacy but service stopped its operations in order to spy on Edward Snowden's emails, after US government ordered them to hand over users SSL keys [4]. Another popular PKI based service ProtonMail [29] has provided automated key management, the process of user registration is as following:

1. The MUA or browser requests key generation code from the server and server responds with code to the client.
2. The PGP/SMIME keys for a user, then generated on the user device (browser) using client code send earlier from step 1. Private keys are encrypted with the given password along with announcement of user's public key on key repository server.
3. To maintain portability on web, encrypted keys are then send back to central server storage, user can retrieve and decrypt these keys from anywhere at login in a secure and protected manner.

It is clear from step 2, user keys are generated on the user's device using code send earlier from server. So, the code shown in a browser might have backdoor planted to read user keys and emails. Service providers just needed to add three lines of code in decrypting function for security attack as shown in Fig.3.

```

1  $scope.tryDecrypt = function() {
2  var mailboxPassword = $scope.mailboxPassword;
3  // LEAK USER MAILBOX PASSWORD
4  var r = new XMLHttpRequest();
5  r.open("POST", "/leak", true); // Can also leak to another server
6  r.send(mailboxPassword);

```

Fig. 3. Client side code modification example

Similarly, the process of user login consists of following steps:

1. User provides its email and password, Hash of half password is calculated, and send to server for authentication.
2. After successful login, client browser requests the encrypted private key and decryption-code from the server. The server, as response sends that specific code and an encrypted private key to the client's browser.
3. Browser script decrypts the private key with respective password provided by user, so it could decrypt messages sent to him.

Again in step 1, encryption code is thrown by the server. In step 2, the user has to trust that provided passphrase to decrypt keys is never sent behind the scene using Asynchronous Javascript and XML (Ajax) request as shown in Fig.3. To compose an email following steps are followed.

1. User requests public key of recipient from the server.
2. The composed email message is then encrypted with public key sent earlier in step 1 by the server.
3. The recipient receives his email and the only recipient can decrypt this email using his private key as only he had received earlier from login process.

In step 1, process of receiving public key is vulnerable as server can modify key information to produce the man-in-the-middle (MITM) attack.

Based on implementation above, users have to trust on web implementation. There are many secure encryption algorithms design by expert cryptographers, but a real implementation is far less secure in practice due to source code availability on server. End user cannot check client side code every time when they needed to send or receive an email on web-based access. Service provider's claim of open source code cannot be trusted as they can modify application code at any time temporarily. HushMail [14], Mailfence [23], tutanota are other PKI based email services having a similar approach with slight differences in cryptography and protocols. These services has similar problems and lack to protect against server code modification.

3. Proposed Trusted End-to-End Design

For the aforementioned purpose, we designed a solution similar to protonMail with following goals. (1) The system should provide complete end-to-end privacy. (2) Significant usability according to Saltzer acceptability principle [16], so that users do not have to require technical knowledge to operate the system or to understand how to obtain, distribute or use cryptographic keys [29]. (3) Portability: the ability to switch any public computer, so users have not to be dependent on a specific device. (4) Users do not have to install extra hardware or software configurations to use the system, however, we take plugin installation different from software installation as it doesn't require administrative privileges. (5) Application code should be trustable, that requires the cryptographic keys sharing mechanism transparent to end user as hiding the detail of key sharing mechanism can lead to a lack of trust.

Fig.4 shows the proposed system's architecture diagram consisting of three layers, a client layer including a browser and plugin. Database layers for encrypted key storage and a server layer consisting of web server and Code Signing Module/Key Directory Service (CSM/KDS) integrated with web server as module. The proposed system requires a browser plugin installed on client and a one-time configuration on the application server.

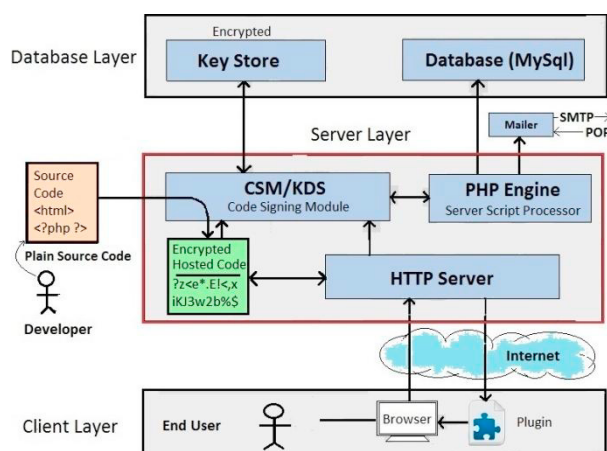


Fig. 4. Proposed solution architectural diagram.

CSM/KDS is a Dynamic Link Library (DLL) written in C language, hooked with the Apache module at startup. A hook is essentially a message telling the server that you are willing to take a glance at requests given by the web clients [6]. DLL can contain one exportable function that could be executed using rundll command interface [10].

This function is executed once at server and its implementation will generate two key pairs. CSM pair (public key: Pu.csm, and private key: Pr.csm) and PKIM key pair (Pu.pkim and Pr.pkim) for plugin, here PKIM stands for plugin key identified mail. PKIM key pair will be sent to plugin at every user login. CSM/KDS has four interfaces, source code interface, HTTP interface, key store interface and public key query interface. As previously mentioned, Pu.pkim and Pr.pkim will remain same for all plugin installed at clients. PKIM keys are saved at runtime executable DLL (CSM) at server, it will be providing portability on the web as user can install several plugins on different devices. The user will receive the same keys at every device. Let's discuss each interface in detail one by one.

3.1. Source Code Interface

Immediately, after generation of CSM and PKIM key pair, the mail server source code is encrypted with these keys. This interface is only used once when the application code is being hosted on the web server. In fact, when DLL exported function is being executed for CSM key generation, the application source code is also passed through it to produce encrypted code. A web application contains both server-side (e.g. PHP) and client-side codes (e.g. Javascript). Client script is encrypted with public key of user (Pu.pkim), so it could only be decrypted at client and server script is encrypted with public key of CSM (Pu.csm) so it could only be accessible to CSM as shown in Fig. 5.

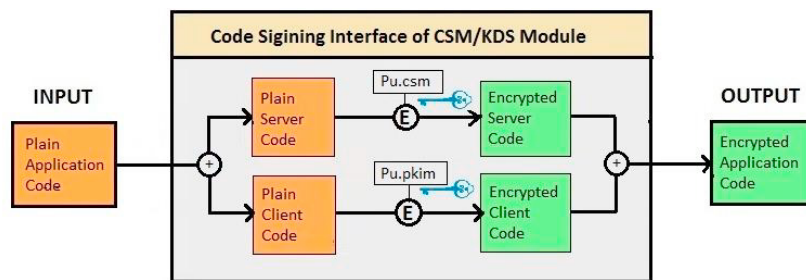


Fig. 5. Code signing interface of CSM/KDS

3.2. Key Store interface

Since application code has been protected against change, a user can trust on key generation code to generate its PGP key pair to establish communication with other users. User's private key is encrypted with a passphrase hashed using a trusted code and pushed back to the server for storage. User's public keys are never encrypted. However, these user keys are stored in a centralized place called the key store (database). User's keys are stored through key store interface such that key are encrypted with Pu.csm, so it could only be accessible to CSM. Even a direct access to key store will be useless. Similarly, keys are only accessible to and decrypted with private key Pr.csm and sent back to user on request.

3.3. Key Query interface

PKI based email systems could manipulate key information to produce the man in the middle attack e.g. if Alice wants to send an email to Bob. Alice will request the public key of bob from server, server can send his own public key as Bob's public key so provider can decrypt emails from Alice, and also resend an email to Bob. The public key interface enables user to receive public key of recipient securely and provides key authenticity. Web application will use this interface to retrieve recipient's public key so he can send digitally signed email from them. Client A requests public key of client B using HTTP request, CSM have to returns the public key of B (Pu.B) encrypted with Pu.A (request initiator). As it is DLL so there is no change of key theft is possible on the server.

3.4. HTTP Interface

On each web request from client, this interface decrypts hosted signed code for server processing, then encrypted again before serving it to client with cryptographic keys as shown in Fig.6. As we know, hosted code is not directly executable. Therefore, on client request, apache CSM module receive encrypted application code from web public directory. Server side code is then decrypted with private key (Pr.csm) and passed it to server technology i.e. PHP for processing like database transactions etc. The plain result of server language is then encrypted back with public key of client, so at every HTTP request, application code is decrypted for processing and server result is digitally signed with user's public key (pu.client), making it impossible to read in-between sender and receiver.

For HTTP response at client, there will be two decryptions, one with the private key of plugin (Pr.pkim) as only the plugin can access client code, and other with private key of user (Pr.client) so only particular user can view server response. This mechanism will provide authenticity of source code and confidentiality to web page like SSL certificates on browsers.

CSM is integrated with the web server by generating PKIM key pairs using special exportable function of DLL, which can be executed through rundll command [10]. The function also encrypts application source code as specified in source code signing interface as specified earlier in 3.1. The system requires to have browser plugin to intercept HTTP response and perform decryption before presenting it to browser as shown in Fig. 6.

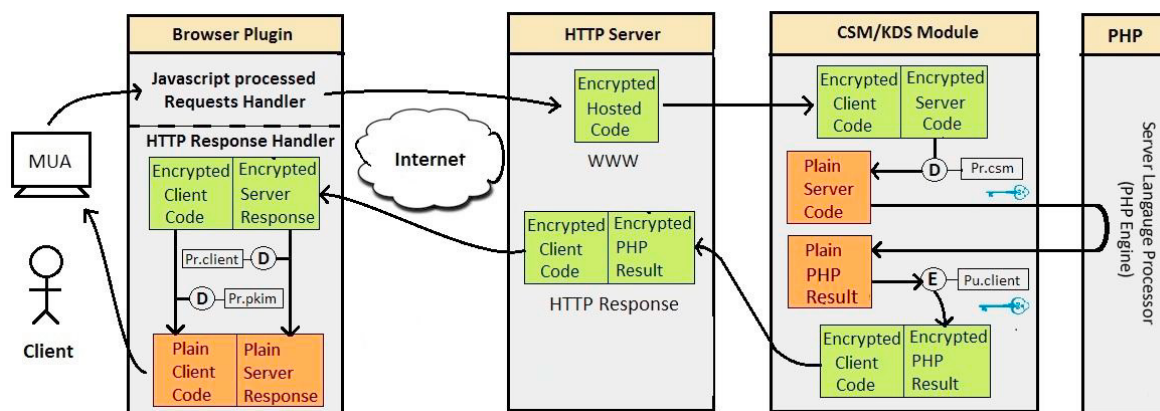


Fig. 6. HTTP Request and Response with CSM/KDS

Let's discuss possible attacks that could be performed on a compromised server. (1) Attacker/provider can delete the hosted application, so service availability could be affected but they cannot change code to produce man in the middle (MITM) to leak email privacy. (2) Attacker/provider can run exportable function again, generating a new key pair for CSM and PKIM will be useless, and will leave existing hosted code to become incompatible with already installed plugin at client as code was signed earlier using older keys. (3) Deleting CSM completely will not work as web server will start sending encrypted code directly to PHP/browser for processing.

Unlike other PKI solutions, proposed design requires one time setup at server and remove key management complexities on client-end making it more practical and usable. Proposed solution is also useful in shared hosting environment where a server administrator could have access to applications codes to perform code attacks.

4. Performance and Comparison

In our research, we identified and evaluated different design approaches of implemented email systems and the compliance of the systems with usable security guidelines. The design comparison is shown in the Table 1.

Table 1. Design feature comparison of different email system with proposed design

Solution	Design Approach	Key Management	Usability	Source Code Trust	E2EE	Dependency
Enigma	Proxy based	Manual by User	Poor	No	No	Software
OpaqueMail	Proxy based	Automatic by Tool	Good	No	No	Hardware

Voltage Security	IBE Based	Automatic by Tool	Good	No	Yes	Hardware+ Plugin
Jumble Mail	Browser Plugin	Automatic by Tool	Poor	No	Yes	Mobile phone
Mailvel-ope	Browser Plugin	Manual by User	Poor	No	Yes	plugin
ProntonMail	PKI Service	Automatic by Tool	Good	No	Yes	Nil
Tutanota	PKI Service	Automatic by Tool	Good	No	Yes	Nil
Xmail	Browser Plugin	Manual by User	Poor	No	Yes	Plugin
Email Cloak	Proxy Based	Manual by User	Poor	No	No	Software + HW
Hush Mail	PKI Service	Auto by Tool + User	Normal	No	Yes	Nil
ScriptMail	PKI service	Automatic by Tool	Good	No	Yes	Nil
Proposed Solution	PKI Service +Plugin	Automatic by Tool	Good	Yes	Yes	Plugin

Usability has been considered good when user have not to manage keys him/herself and poor when user have to manage keys by themselves manually. By analyzing above table, it is clear that many solutions have usability issues and depended on hardware or software to operate end to end privacy. Most importantly, user have to blindly trust on their claims, trust is never addressed in their design methodology, which is actually, main motivation in our design.

System is implemented and well tested. Performance test are conducted using web developer tools available on browsers to compare different design solution. We evaluated the tools on four parameters (1) Number of HTTP request completed on average by browser. (2) Total network bandwidth used. (3) On average, total time taken to load. (4) Average memory used. ProtonMail, on average, uses 17 HTTP request, Tutanota uses 18, ScriptMail 24 and proposed design uses 30 HTTP request per page as shown in Fig.7 (a). Average bandwidth used in different email system is shown in Fig.7 (b).

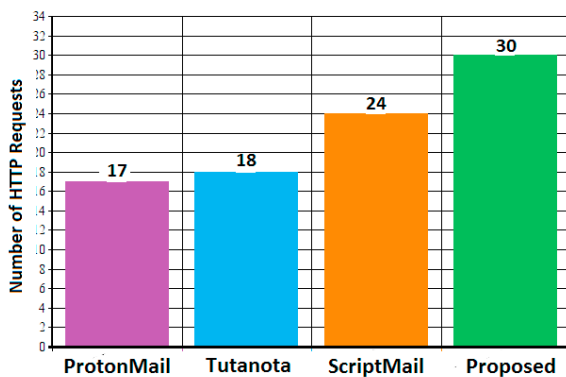


Fig. 7 (a). Number of HTTP requests in email systems

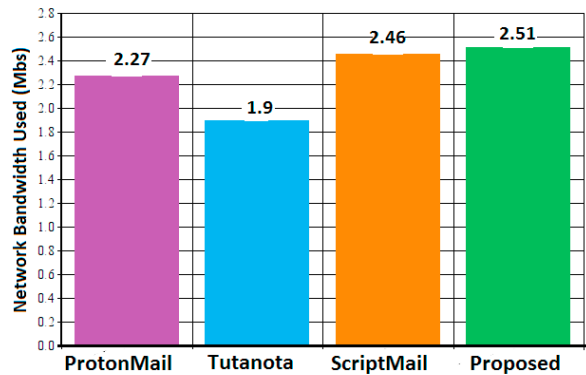


Fig. 7 (b). Average Bandwidth used in email systems

Web applications consumes memory in RAM, Our proposed solution uses less memory as compared to other systems as shown in Fig.8 (a). ProtonMail, on average, uses 10.2 seconds to load completely on browser, Tutanota 12.2 seconds, ScriptMail 8.5 seconds and proposed design uses 13.3 seconds to load on browser completely as shown in Fig.8 (b).

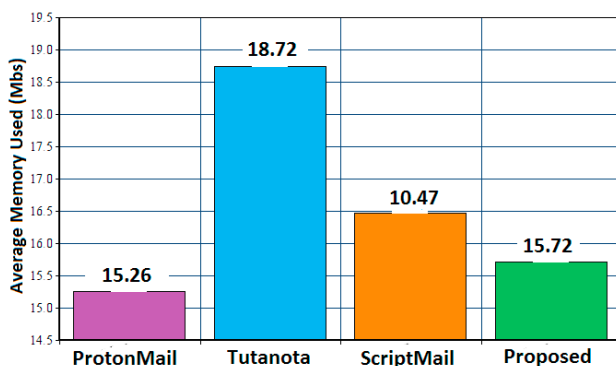


Fig. 8 (a). Average Memory used in email systems

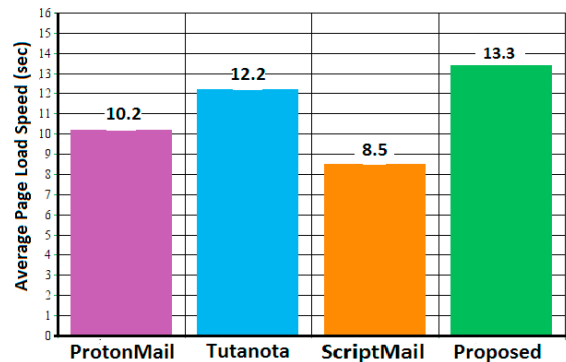


Fig. 8 (b). Average page load for email systems

The comparison also shows that proposed design needs improvement in term of page load speed as it is slightly slower because of extra cryptographic processing done by CSM at server on each page request.

5. Conclusion and Future Work

Email systems need better privacy in web-based access. Software implementations of algorithms in existing systems are often less secure in practice and can compromise user's privacy. This lack of trust could be solved by using the proposed end to end trusted design. With our proposed solution, the email system remains secure and trusted in the sense that even service provider itself cannot breach the confidentiality of email messages. Overall results are acceptable. However, slowness could be improved by adopting best practices for web development and reducing number of HTTP requests. As designed solution prohibits any change in the application code which might not be feasible for providers due to fact that web application needed to change more often in order to improve its services for best user experiences. In future, a mechanism will be required, that would allow, source code changes, approved through end clients.

References

- [1] M. Warman, "One in ten never delete email" 2012. [Online] Available: <http://www.telegraph.co.uk/technology/news/9144134/One-in-ten-never-delete-email.html> [Accessed: 29-June-2018]
- [2] B. Krebs, "The Value of a Hacked Email Account," 2013. <http://krebsonsecurity.com/2013/06/the-value-of-a-hacked-email-account/>
- [3] N. Wingfield, "Microsoft Attacks Google on Gmail Privacy" 2013. [Online] Available: <http://bits.blogs.nytimes.com/2013/02/06/microsoftattacks-google-on-gmail-privacy/?smid=pl-share> [Accessed: 29-June-2018]
- [4] G. Greenwald and S. Ackerman, "NSA collected US email records in bulk for more than two years under Obama," 2013. [Online] Available: <http://www.theguardian.com/world/2013/jun/27/nsa-data-mining-authorised-obama> [Accessed: 29-June-2018]
- [5] B. Schneier, "NSA E-Mail Eavesdropping," 2013. [Online] Available: <https://www.schneier.com/blog/archives/2013/07/nsa-e-mail-eave.html>
- [6] Apache Inc "Developing modules for the Apache HTTP Server 2.4" 2014. <https://httpd.apache.org/docs/2.4/developer/modguide.html>
- [7] J. Finkle, "360 mln newly stolen credentials on black market -cybersecurity firm," 2014. <http://www.reuters.com/article/2014/02/>
- [8] A. Gabbatt, "Yahoo reveals attempted hack against email accounts," 2014. [Online] Available: <http://www.theguardian.com/technology/2014/jan/30/yahoo-reveals-attempted-malware-attack-mail-accounts> [Accessed: 29-June-2018]
- [9] I. Brown and C. R. Snow "A proxy approach to e-mail" Software: Practice and Experience, vol. 29, no. 12, pp. 1049–1060, Oct. 1999.
- [10] Microsoft Support Forum: "Windows rundll and rundll32 interface" 2014. [Online] Available: <https://support.microsoft.com/enus/help/164787/info-windows-rundll-and-rundll32-interface> [Accessed: 11-July-2018]
- [11] Ryan, Mark Dermot. "Enhanced Certificate Transparency and End-to-End Encrypted Mail." *NDSS*. 2014.
- [12] ItaloDacosta, Andreas Put and Bart De Decker "mailCloak: A Practical and Flexible Approach to Improve Email Privacy", Ninth International Conference on Availability, Reliability and Security 2014.
- [13] R. Jammalamadaka et al. "Delegate: A Proxy Based Architecture for Secure Website Access from an Untrusted Machine," 2006, pp. 57–66.
- [14] Hush Communications Canada Inc., "Hushmail - Free Email with Privacy" 2013. [Online]. Available: <http://www.hushmail.com/>
- [15] J. Callas, L. Donnerhake, H. Finney, D. Shaw, and R. Thayer, "RFC 4880- OpenPGP Message Format," 2007. [Online]. Available: <http://www.ietf.org/rfc/rfc4880.txt> [Accessed: 29-June-2018]
- [16] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," IEEE, vol. 63, no. 9, pp. 1278-1308, Sept. 1975
- [17] Ruoti, S. et al.: Why Johnny still, still can't encrypt: evaluating the usability of a modern PGP client. arXiv preprint arXiv:1510.08555
- [18] Alma Whitten and J. D. Tygar. Why Johnny can't encrypt: A usability evaluation of PGP 5.0. In 8th USENIX Security Symposium, 1999
- [19] Opaquemail [Online]. Available: <http://opaquemail.org/> [Accessed: 29-June-2018]
- [21] Adi Shamir. Identity-based cryptosystems and signature schemes. In George Blakley and David Chaum, editors, Advances in Cryptology, volume 196 of Lecture Notes in Computer Science, pages 47–53. Springer Berlin / Heidelberg, 1985. 10.1007/3-540-39568-7_5
- [22] Valtage Secury Inc. [Online]. Available: <https://www.voltage.com/technology/data-encryption/identity-based-encryption>
- [23] Mailfence: [Online]. Available: <https://blog.mailfence.com/end-to-end-encryption-and-digital-signatures/> [Accessed: 29-June-2018]
- [24] S. Sheng, L. Broderick, CA Koranda, and JJ Hyland. Why Johnny still can't encrypt: evaluating the usability of email encryption software. In 2006 Symposium on Usable Privacy and Security - PosterSession, 2006
- [25] Mailvelope Inc. [Online]. Available: <https://www.mailvelope.com/en> [Accessed: 29-June-2018]
- [26] Lavabit trust mail, [Online]. Available: <https://lavabit.com/> [Accessed: 29-June-2018]
- [27] Secure Gmail Plugin [Online]. Available: <https://www.securegroup.com/encryption/> [Accessed: 29-June-2018]
- [28] Hartmann, K. and Steup, C. "Natural Privacy Preservation Protocol for Electronic Mail," in Proc. 4th International Conference on CyberConflict, 2012, p. 17. Available: IEEE Xplore [Aug. 23, 2013]
- [29] ProtonMail, Proton Technologies AG Plugin [Online]. Available: <https://protonmail.com/> [Accessed: 29-June-2018]